



Práctica 04: Refactor a Service Layer



Objetivo

Refactorizar una API de endpoints "gordos" a una arquitectura con **Service Layer**.



Escenario

Tienes una API con toda la lógica en los endpoints. Tu tarea es:

1. Crear un Service para Authors
2. Crear un Service para Posts
3. Refactorizar los endpoints para usar los Services



Estructura Objetivo

```
starter/
├─ main.py           # App FastAPI + routers
├─ database.py       # Engine + Session
├─ models.py         # SQLAlchemy models
├─ schemas.py        # Pydantic schemas
├─ services/         # 🧳 NUEVA CARPETA
│   ├── __init__.py
│   ├── author_service.py
│   └── post_service.py
└─ routers/          # 🌐 Endpoints refactorizados
    ├── __init__.py
    ├── authors.py
    └── posts.py
```



Instrucciones

Paso 1: Revisar el Código Actual

Abre `starter/main_before.py` y observa el endpoint "gordo":

```
@app.post("/posts")
def create_post(post: PostCreate, db: Session = Depends(get_db)):
    # Validación de negocio
    author = db.get(Author, post.author_id)
    if not author:
        raise HTTPException(404, "Author not found")

    # Procesar tags
    tags = []
    for tag_name in post.tag_names:
        tag = db.execute(select(Tag).where(Tag.name == tag_name)).scalar()
        if not tag:
            tag = Tag(name=tag_name)
            db.add(tag)
            tags.append(tag)

    # Crear post
```

```
db_post = Post(title=post.title, content=post.content, author_id=post.author_id, tags=tags)
db.add(db_post)
db.commit()
db.refresh(db_post)
return db_post
```

Problemas identificados:

- 🍷 Lógica de negocio mezclada con HTTP
- 🍷 Difícil de testear sin HTTP
- 🍷 No reutilizable

Paso 2: Crear AuthorService

Abre `starter/services/author_service.py` y descomenta:

```
class AuthorService:
    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, author_id: int) -> Author | None:
        return self.db.get(Author, author_id)

    def get_by_email(self, email: str) -> Author | None:
        stmt = select(Author).where(Author.email == email)
        return self.db.execute(stmt).scalar_one_or_none()

    def list_all(self, skip: int = 0, limit: int = 10) -> list[Author]:
        stmt = select(Author).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

    def create(self, data: AuthorCreate) -> Author:
        if self.get_by_email(data.email):
            raise ValueError(f"Email {data.email} already exists")

        author = Author(name=data.name, email=data.email)
        self.db.add(author)
        self.db.commit()
        self.db.refresh(author)
        return author

    def delete(self, author_id: int) -> None:
        author = self.get_by_id(author_id)
        if not author:
            raise ValueError(f"Author {author_id} not found")

        self.db.delete(author)
        self.db.commit()
```

Paso 3: Crear PostService

Abre `starter/services/post_service.py` y descomenta:

```
class PostService:
    def __init__(self, db: Session):
```

```

self.db = db

def get_by_id(self, post_id: int) -> Post | None:
    stmt = (
        select(Post)
        .options(joinedload(Post.author), selectinload(Post.tags))
        .where(Post.id == post_id)
    )
    return self.db.execute(stmt).scalar_one_or_none()

def create(self, data: PostCreate) -> Post:
    # Validar autor
    author = self.db.get(Author, data.author_id)
    if not author:
        raise ValueError(f"Author {data.author_id} not found")

    # Procesar tags
    tags = self._get_or_create_tags(data.tag_names or [])

    # Crear post
    post = Post(
        title=data.title,
        content=data.content,
        author_id=data.author_id,
        tags=tags
    )
    self.db.add(post)
    self.db.commit()
    self.db.refresh(post)
    return post

def _get_or_create_tags(self, names: list[str]) -> list[Tag]:
    tags = []
    for name in names:
        tag = self.db.execute(
            select(Tag).where(Tag.name == name)
        ).scalar_one_or_none()
        if not tag:
            tag = Tag(name=name)
            self.db.add(tag)
        tags.append(tag)
    return tags

```

Paso 4: Refactorizar Router de Authors

Abre `starter/routers/authors.py` y descomenta:

```

@router.post("/", response_model=AuthorResponse, status_code=201)
def create_author(author: AuthorCreate, db: Session = Depends(get_db)):
    service = AuthorService(db)
    try:
        return service.create(author)
    except ValueError as e:
        raise HTTPException(status_code=409, detail=str(e))

```

```
@router.get("/{author_id}", response_model=AuthorResponse)
def get_author(author_id: int, db: Session = Depends(get_db)):
    service = AuthorService(db)
    author = service.get_by_id(author_id)
    if not author:
        raise HTTPException(status_code=404, detail="Author not found")
    return author
```

Paso 5: Refactorizar Router de Posts

Abre `starter/routers/posts.py` y descomenta:

```
@router.post("/", response_model=PostResponse, status_code=201)
def create_post(post: PostCreate, db: Session = Depends(get_db)):
    service = PostService(db)
    try:
        return service.create(post)
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e))

@router.get("/{post_id}", response_model=PostResponse)
def get_post(post_id: int, db: Session = Depends(get_db)):
    service = PostService(db)
    post = service.get_by_id(post_id)
    if not post:
        raise HTTPException(status_code=404, detail="Post not found")
    return post
```

Paso 6: Actualizar main.py

```
from fastapi import FastAPI
from database import engine, Base
from routers import authors, posts

Base.metadata.create_all(bind=engine)

app = FastAPI(title="Blog API con Service Layer")

app.include_router(authors.router)
app.include_router(posts.router)
```

Paso 7: Probar la API

```
cd starter
uvicorn main:app --reload
```

Prueba en `/docs` :

1. Crear un autor
2. Crear un post con tags

✓ Checklist de Verificación

- ☐ Services creados en carpeta `services/`
 - ☐ Routers refactorizados para usar Services
 - ☐ Lógica de negocio está en Services, no en Routers
 - ☐ Services lanzan excepciones Python, Routers las convierten a HTTP
 - ☐ La API funciona igual que antes del refactor
-

🎯 Reto Extra

1. Agrega método `update()` a `AuthService`
 2. Agrega método `add_tag()` y `remove_tag()` a `PostService`
 3. Crea tests unitarios para los Services (sin HTTP)
-

[← Anterior: Práctica 03](#) | [Volver al README →](#)